

JSON Parsers interoperability

The freedom punishment

Daniel Haro I Miquel Rius



\$ whoami

Daniel Haro

```
{  
  "me": "Pentester en Var Group España",  
  "me": "Web hacking lover",  
  "me": "Solía jugar CTFs, hacer  
         researching y escribir en mi blog",  
  "me": "\\0",  
}
```



- whoissecure.xyz
- [@whoissecure](https://twitter.com/whoissecure)

\$ whoami

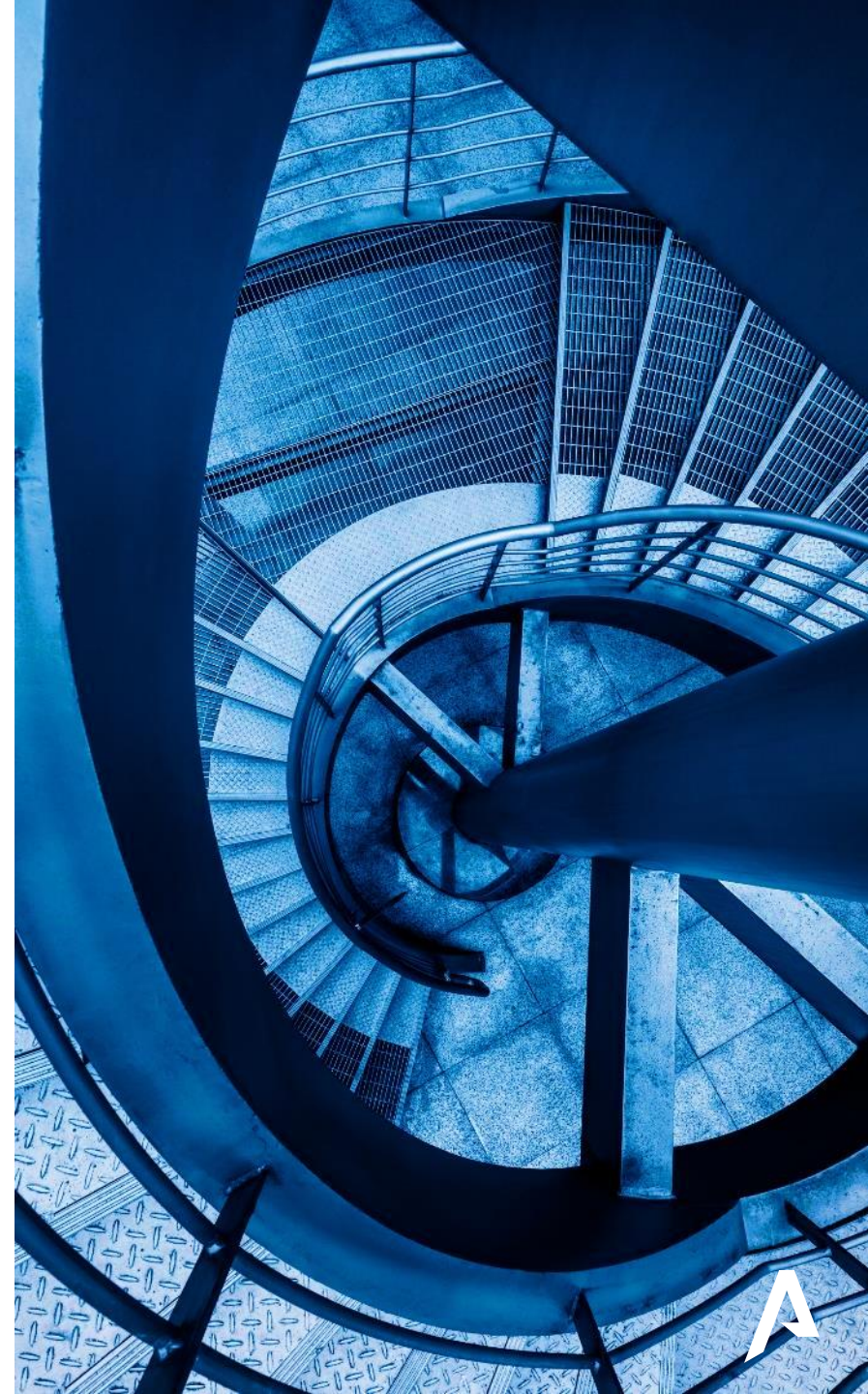
Miquel Rius

```
{  
  "me": "Pentester en Var Group España",  
  "me": "Reversing amateur",  
  "me": "WoW enjoyer",  
  "me": "\0",  
}
```



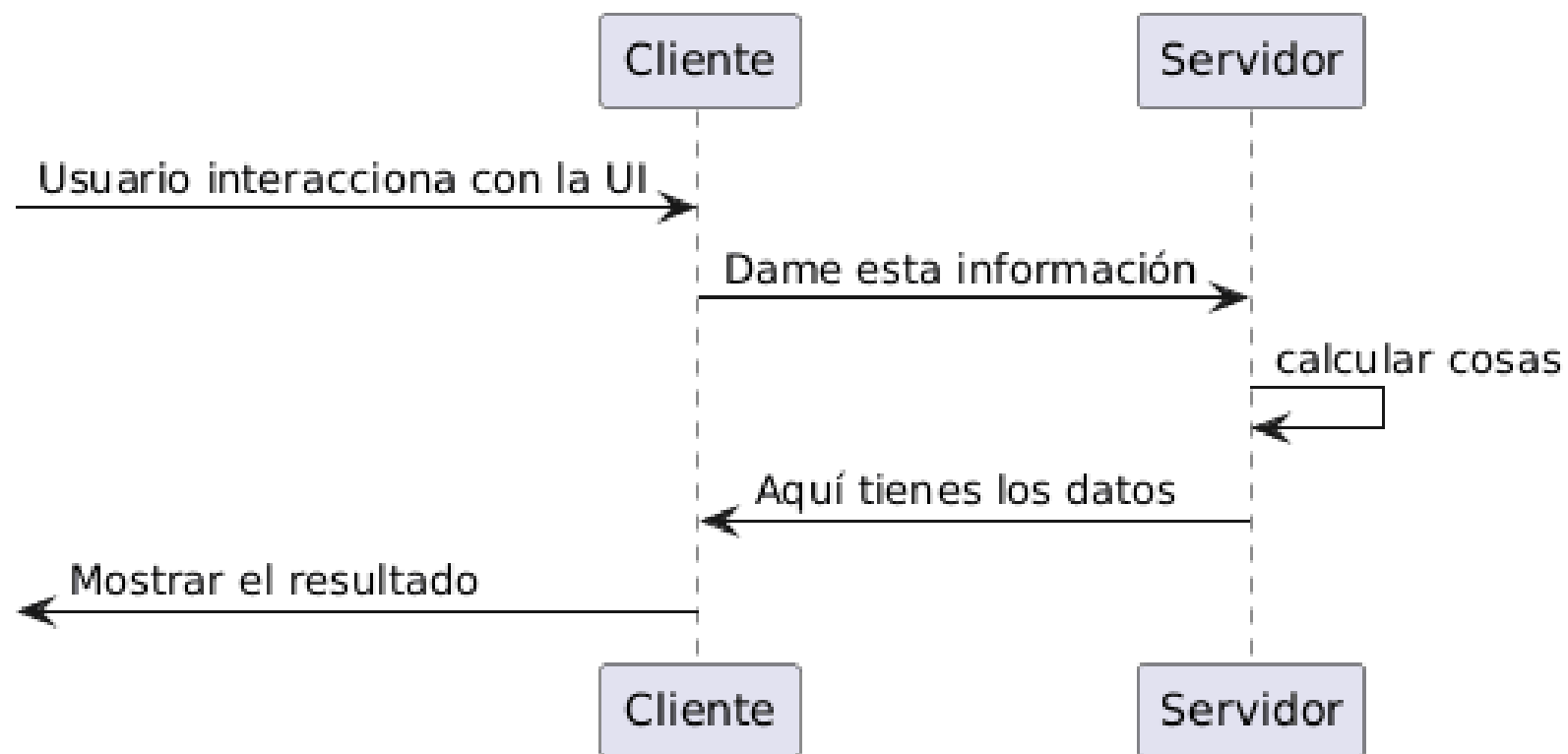
Introducción

- ¿Qué es JSON?
- IETF RFC 8259: Donde empiezan los problemas
- Problemas comunes en los parsers
- Un vistazo al CVE-2017-12635
- Explotando estas discrepancias: Momento demo
- Recomendaciones

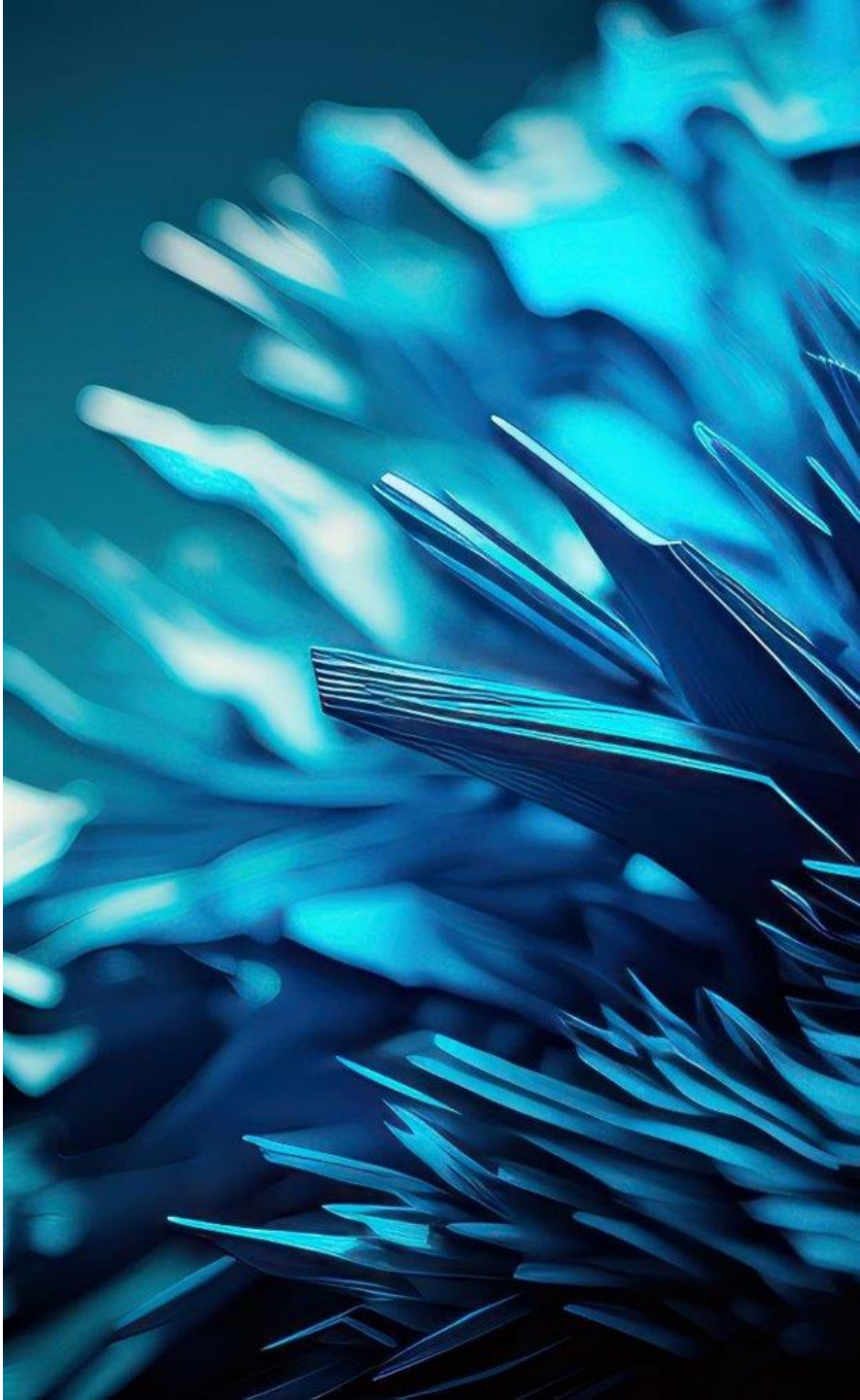
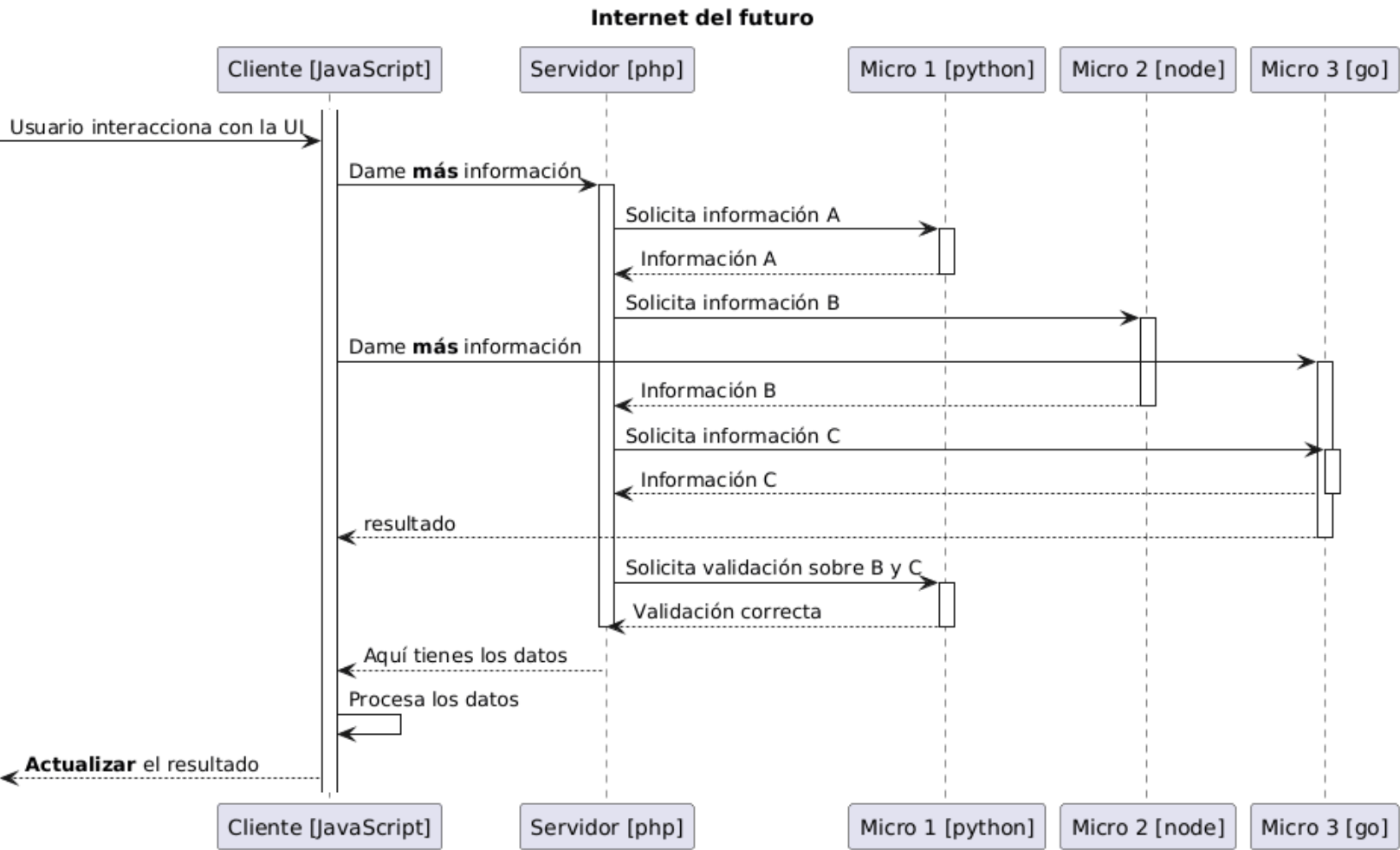


¿JSON?

Internet del pasado



¿JSON?



JavaScript Object Notation

- Nace en 2001
- **Soluciona**
 - Poder tener una sesión con estado y no necesitar refrescar
 - Compartir información entre diferentes componentes
- **Lo hace muy bien**
 - Estándar en aplicaciones móviles y microservicios
 - Todo el mundo lo implementa



JavaScript Object Notation

- **Cuatro tipos de datos**
 - Números
 - Strings
 - Booleans
 - Null
- **Dos estructuras**
 - Pares clave/valor {“k”:”v”}
 - Lista ordenada de valores [1,6,2]

Ejemplo

string	This is a string
number	42
bool	☒ true
null	☐
array	•
object	•

text in array
123
☐ false
☐
•

nestedString	Nested value
nestedNumber	3.14

nestedObject	value
---------------------	-------



Regulando JSON

- JSON.org
- ECMA 262
- **ECMA 404**
- **ISO/IEC 21778:2017**
- RFC 4627
- RFC 7159
- **RFC 8259**



Empiezan los problemas: Números

- No existen los enteros
- Implementaciones deciden el rango
- Se espera mínimo IEEE754 double-precision (64 bits float)
 - $[-(2^{53})+1, (2^{53})-1]$
- No hay **Infinitos**
- No hay **NaN**

This specification **allows implementations to set limits** on the range and precision of numbers accepted.

Numeric values that cannot be represented in the grammar below (such as **Infinity** and **NaN**) are not permitted.



Empiezan los problemas: Strings

- UTF-8 (a veces)
 - Si no es un sistema cerrado
 - Si es RFC-8259 compliant
- Sin BOM (pero pueden ignorarlo)
- Implementaciones deciden en los límites de los tamaños y caracteres que soportan.

{
An implementation may set
limits on the size of texts
that it accepts.
}

{
An implementation may set limits on
the length and character contents
of strings.
}



Empiezan los problemas: Strings: UNICODE

- Es **UNICODE**
 - Transformaciones
 - Comparaciones de strings
 - Valores que no son unicode válidos

this specification allows member names and string values to contain bit sequences that cannot encode Unicode characters; for example, "\uDEAD".

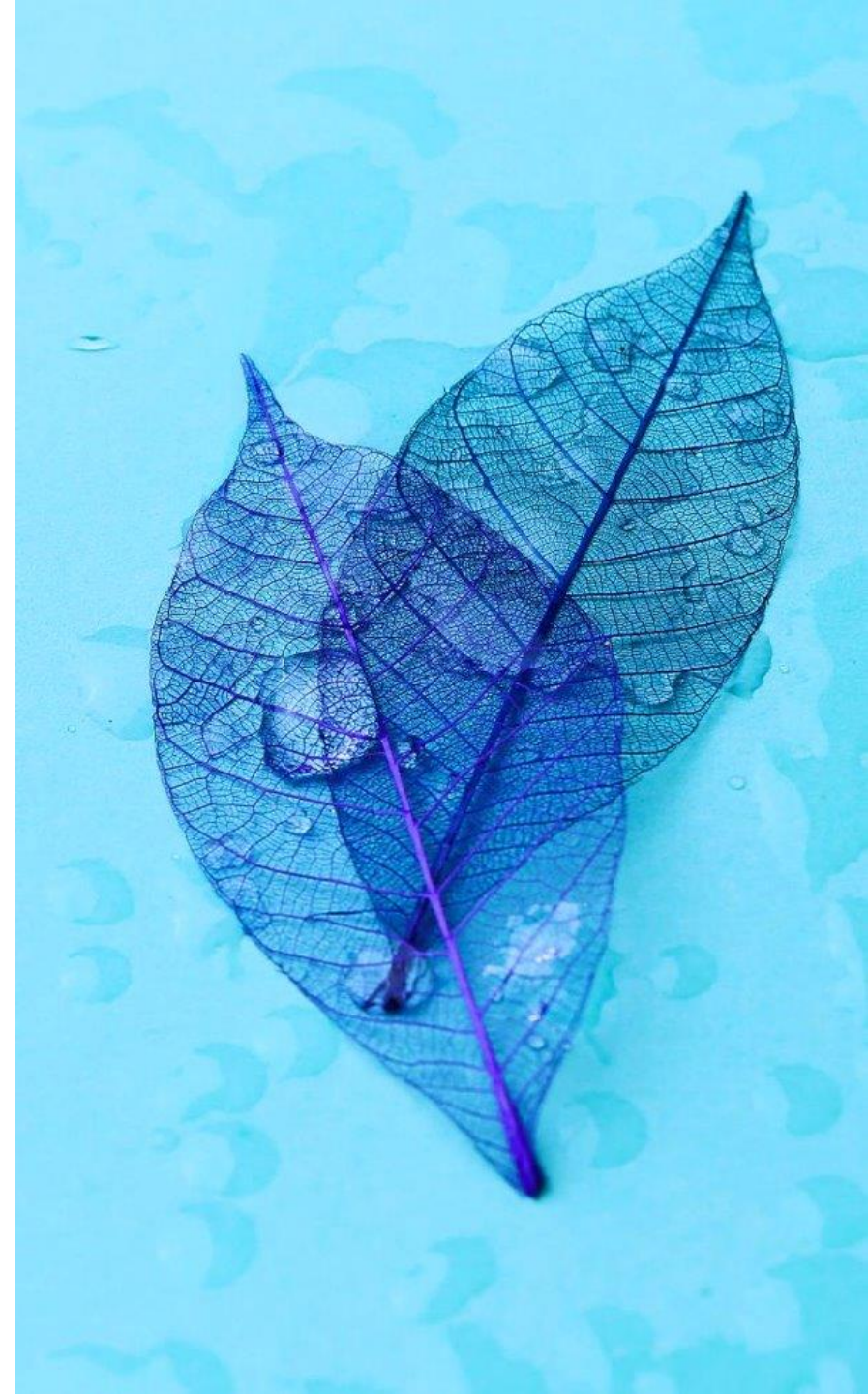
The behavior of software that receives JSON texts containing such values **is unpredictable**.



Empiezan los problemas: Objetos y Listas

- Implementaciones deciden qué hacer con claves duplicadas
- Implementaciones deciden si preservar el orden de las claves
- Implementaciones deciden en los límites de profundidad: nesting

{ When the names within an object are not unique,
the behavior of software that receives such an
object is **unpredictable**. }



Discrepancias comunes peligrosas

- Duplicado de keys
- Type confusion
- Normalización Unicode
- Bugs de lógica que den lugar a DoS
- Comentarios



Un vistazo al CVE-2017-12635

Apache CouchDB Privilege Escalation

- Erlang parser vs JavaScript-based parser
- Claves duplicadas
 - { "type": "user", "name": "oops", "roles": ["_admin"], "roles": [], "password": "password" }
- Obtención del primer valor en Erlang con:
 - `lists:keysearch(Key, 1, List).`

```
> JSON.parse("{\"foo\":\"bar\", \"foo\": \"baz\"}")  
{foo: "baz"}
```

```
> jiffy:decode("{\"foo\":\"bar\", \"foo\": \"baz\"}").  
{[{<<"foo">>, <<"bar">>}, {<<"foo">>, <<"baz">>}]}
```





<https://github.com/whoissecure/talks>



Momento demo

Duplicado de clave

- Golang buger/jsonparser vs Python default parser
- Discrepancia parseando
 - `{"user":"daniel", "password":"1234", "role":"moderator", "role":"admin"}`
- Obtención del primer valor en la API en Go y del último en la API

Python



Momento demo

```
1 package main
2 ...
3 ...
4
5 var users = map[string]struct {
6     ...
7 }{
8     "daniel": {"1234", []string{"moderator", "viewer"}},
9 }
10
11 ...
12
13 func loginHandler(w http.ResponseWriter, r *http.Request) {
14     ...
15     if !contains(u.Roles, role) {
16         http.Error(w, `{"error": "role not allowed"}`, http.StatusForbidden)
17         return
18     }
19     resp, err := http.Post("http://lab1-python:5000/internal",
20         "application/json",
21         bytes.NewReader(body))
22     ...
23 }
24
25 func main() {
26     http.HandleFunc("/login", loginHandler)
27     fmt.Println("Go backend listening on :8080")
28     log.Fatal(http.ListenAndServe(":8080", nil))
29 }
```

```
1 from flask import Flask, request, jsonify
2 import jwt
3
4 app = Flask(__name__)
5 SECRET = "supersecret"
6
7 @app.route("/internal", methods=["POST"])
8 def internal():
9     try:
10         data = request.get_json(force=True)
11     except Exception:
12         return jsonify({"error": "Invalid JSON"}), 400
13
14     token = jwt.encode({
15         "user": data.get("user", ""),
16         "role": data.get("role", "")
17     }, SECRET, algorithm="HS256")
18
19     return jsonify({"token": token}), 200
```



Momento demo

Uso de NaN

- Python default parser
- Parseo de valor numérico prohibido
 - `{"user":"daniel","quantity":NaN}`
- Bug causado por el uso de NaN en una comparación que siempre tendrá resultado False

```
● ● ●  
>>> import json  
>>> q = json.loads('{"q":NaN}').get("q")  
>>> balance = 20  
>>> q*20 > balance  
False  
>>> q*20 < balance  
False  
>>>
```



Momento demo

```
1 from flask import Flask, request, jsonify
2
3 app = Flask(__name__)
4
5 ...
6
7 @app.route("/buy", methods=["POST"])
8 def buy():
9     try:
10         data = request.get_json(force=True)
11     except Exception:
12         return jsonify({"error": "Invalid JSON"}), 400
13
14     try:
15         total = PRODUCT_PRICE * data.get("quantity")
16         user = data.get("user")
17     except Exception:
18         return jsonify({"error": "Error con los valores recibidos"}), 400
19
20     if total > wallets[user]:
21         return jsonify({"error": "El saldo es insuficiente"}), 200
22     else:
23         wallets[user] = wallets[user] - total
24         return jsonify({"msg": "Compra realizada correctamente"}), 200
25
26 ...
```



Momento demo

Integer overflow

- Perl default parser
- Entero demasiado grande -> Overflow a NaN
 - { "user": "daniel", "quantity": 1.0e9129 }
- Un entero demasiado grande se ve parseado como NaN, y como antes, se usa en una comparación haciendo que esta siempre tenga como resultado False

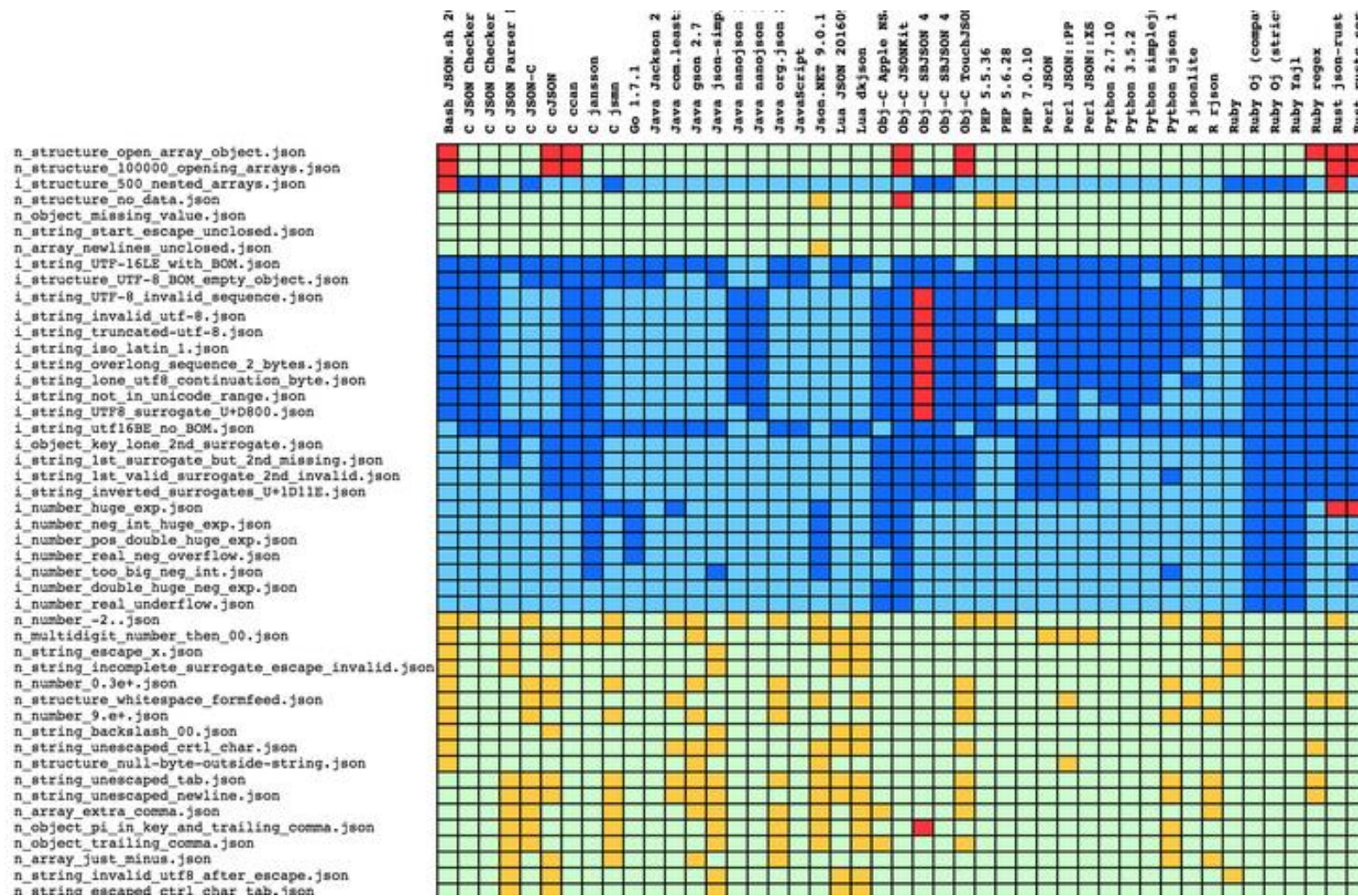


Recomendaciones

- Si mantienes un parser: hazlo RFC en mano, ante la duda, devuelve errores
- Si usas distintos parsers en tu proyecto: reduce el número de parsers, haz inventario, compara y realiza validaciones (muchas validaciones)
- Si eres pentester/bug hunter: suerte



Un vistazo a las diferencias



<https://github.com/Wise-Security/JSONTestSuite/>



¡Gracias por
vuestra atención!

